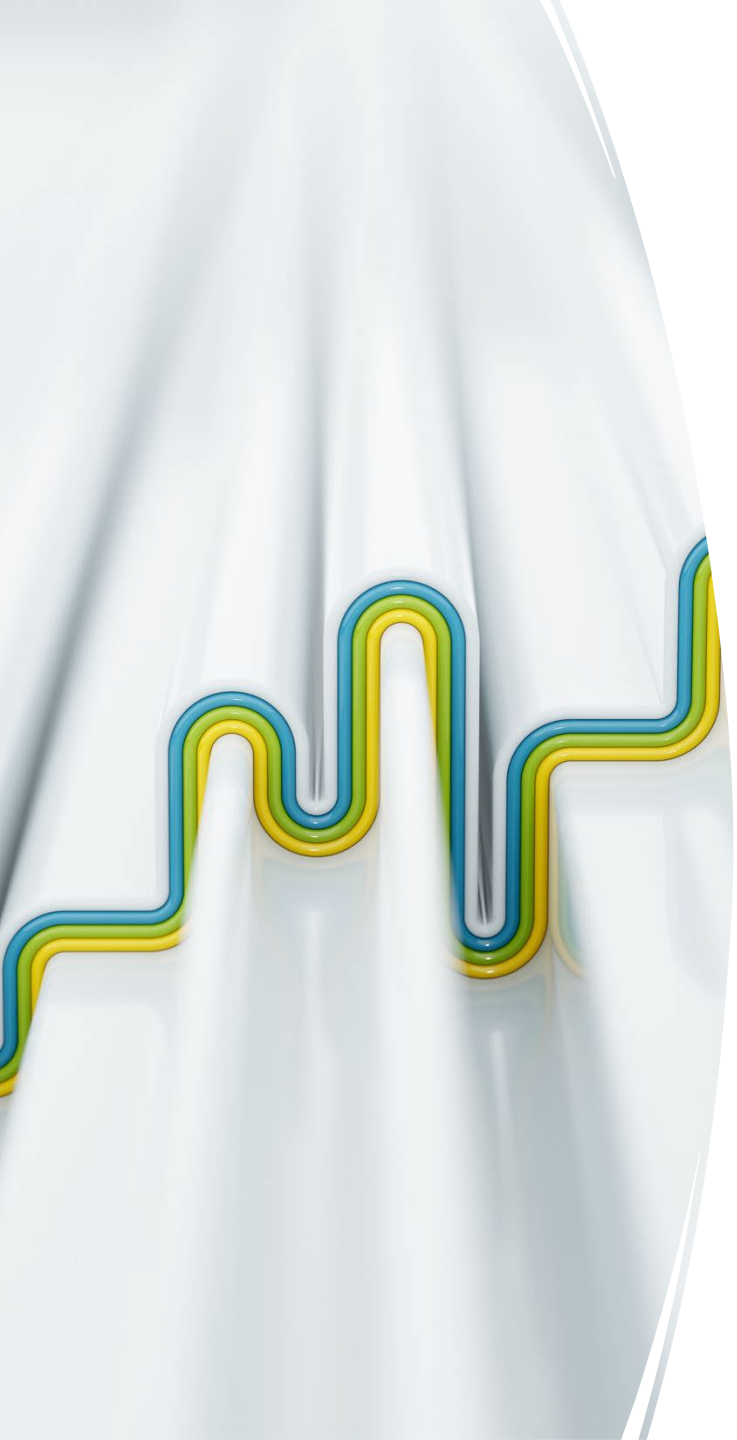


Design & Implementation of a Machine Learning Pipeline

Mid-module check-in

Alex Rush, Juzer Pakawala, Michael Wagg, Rishabh Panikar

Group A4

Project Objective (Rishabh)

- Predict Ranked Solo/Duo match outcome (blue_side_win)
- Use champion draft only
- Can be used to teach newer players the best team comps and strategies.
- Can provide insight to game developers for any balancing issues.

Dataset Choice & Justification (Alex)

- __init__.py
- collect_ids_multi.py
- collect_matches.py
- collect_multi.py
- convert_seeds_to_riotids.py
- debug_api.py
- example_fetch_and_save.py
- feature_engineering.py
- fetch_data.py
- fetch_from_puuids.py
- health_check.py
- ingest.py
- prepare_features.py
- seed_puuids_from_league.py
- train_model.py

- Riot Match V5 Data (Collected via fetch_data.py and collect_matches,py)
- Holds match level records (teams, champions, roles, timestamps)
- Goal was to give team compositions based on per patch strength and winrate
  - Labels are simple win/loss and supervised learning is straightforward, and features are highly predictive for win probability
  - Catergorical and numerical objectives and engineered features from prepare_features.py/feature_engineering.py allowing for both simple and complex models

  The match outcome is objective we control for noise with filtering and aggregate stats per champion to help with rare champion variances. We've also accounted fir patch/version drift, popularity imbalance (weighting) and skill level skew.

Data Storage & Modelling(Juzer)

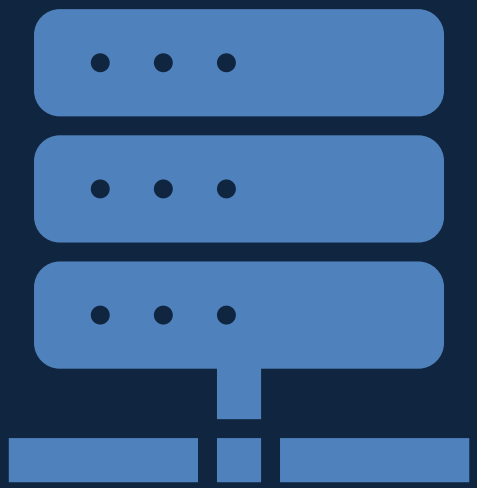
- **Extract** match data from Riot Match V5 API using collection scripts.
- **Store raw responses** as JSON in `data/raw/` as parquet for traceability and reproducibility.
- **Validate and ingest** raw data into MariaDB ColumnStore or staging tables using Python and SQLAlchemy-style loading patterns similar to the labs.
- **Transform** the data using preprocessing and feature engineering scripts to clean missing values, flatten nested fields, encode categorical variables, aggregate team/champion statistics, and generate the target label.
- **Store processed outputs** in `data/processed/` as Parquet/CSV and optionally in curated analytical tables.
- **Orchestrate the stages** with Airflow as DAG tasks for ingestion, preprocessing, training, and evaluation.
- **Pass small metadata** such as file paths or Redis keys via XCom, while using Redis + PyArrow for larger intermediate data if needed.
- **Train and evaluate models**, then persist the best model and metrics in `models/`.

Pipeline Architecture (Alex)

- Data collected from Riot API to gather and store match and player data in the raw folder (.paraquet)
- Ingest.py is used to confirm and persist raw files allowing us to keep raw and processed data separate
- Prepare_feature.py and feature_engineering.py used to clean, join, encode, and aggregate per-team/champion features. Also handles missing values and produces a .paraquet or .csv.
- We are using a time-based split and stratification on roles/champions where proper. We are also keeping a held-out test set for out final model
- We are experimenting with models using train_model.py (XG,LGBM,RF looking at possible neural nets if within scope) using grid or Optuna for hyperparameter tuning. We persist the best model and metrics.json.

Tools & Technologies (Rishabh)

- Python, requests, pandas, pyarrow, xgboost, scikit-learn
- multiprocessing + checkpointing
- JSON used to store raw file data
- Pyarrow is a column-oriented data format designed for fast data exchange.
- Parquet (column-based file format) is used for storage
- Xgboost is used to better help analyse the huge dataset



Progress to Date (Alex)

- Riot fetch API
- 2-3 notebooks of early models
- Data preparation and feature engineering
- Data ingestion and cleaning done
- 5000 matches of data (Collected)
- Xgb_patch_model.pkl (In progress)
- 5 CSV files of per role popularity and wins of ALL champions played

Risks & Ethical Considerations (Michael)

- API rate limits and key expiry
  - Our pipeline relies heavily on external Infrastructure
  - If we exceed API rates, we could risk temporary bans resulting in damage to our dataflow
  - This risk is also carried by a possible API key expiry
- Player privacy — encrypted IDs only
  - Under GDPR data must remain pseudo-anonymous but we can utilise encrypted ID's
  - There will be no attempts to deanonymise any player information
  - Only the usage of encrypted ID's and aggregated statistics, no usernames or personal information.
- Sampling bias & compliance
  - Bias could lead to predictions being misleading
  - This could happen as a result of overrepresentations of champions due to their popularity
  - Minor inherent Blue Side bias will also have to be accounted for

Next Steps & Timeline (Michael)

1

Collect 5k match IDs and fetch JSONs

2

Run full training and tuning

- Logistical Regression (Rishabh)
- Random Forest (Michael)
- XG Boosting (Alex)

3

Containerize, monitor and Deploy

- (Juzer, Alex)

Q&A

- Questions?